THOUGHT PROCESS

Action: Now I have all the context:

Design Guidelines Summary:

- Theme: Warm, earthy coffee shop vibe
- Colors: Terracotta primary (#C05E47), warm backgrounds (#FAF8F5), natural accents
- Typography: Cormorant Garamond (headings), Manrope (body)
- Layout: Tetris grid for marketing, dashboard bento for admin
- High-quality images provided for hero, products, backgrounds

Technical Stack:

- Backend: FastAPI + MongoDB (already set up with basic template)
- Frontend: React + Tailwind + Shadcn components
- Auth: JWT-based (need to implement)

Features from ASP.NET Project to Implement:

1. User Management: Registration, Login, Profile
2. Product Catalog: Browse 12 products (coffee, tea, pastries), search, filter
3. Order Management: Create orders, view history, track status
4. Payment & Invoicing: Mock payment, invoice generation
5. Dashboard: Order history, profile management

Database Models (converting from SQL Server to MongoDB):

- customers: id, name, email, password (hashed), contactNo, address, createdDate, isActive
- products: id, name, description, price, quantity, category, imageUrl, isAvailable, createdDate
- orders: id, customerId, orderDate, subTotal, tax, deliveryCharge, totalAmount, status, deliveryAddress, specialNotes

- orderItems: id, orderId, productId, quantity, unitPrice, lineTotal

- bills: id, orderId, subTotal, tax, discount, totalAmount, billDate, paymentMethod, paymentStatus, transactionId, paymentDate

Sample Data from SQL:

- 12 products: Espresso (₹50), Cappuccino (₹80), Latte (₹75), Americano (₹60), Chai (₹40), Green Tea (₹45), Brownie (₹35), Cake (₹40), Muffin (₹30), Croissant (₹35), Iced Coffee (₹70), Mocha (₹90)

- 3 sample users

Implementation Plan:

1. Install dependencies (bcrypt for password, jwt)

2. Backend (server.py):

   - Auth routes: /api/auth/register, /api/auth/login, /api/auth/profile

   - Product routes: /api/products (list, search, filter), /api/products/:id

   - Order routes: /api/orders (create, list, cancel)

   - Bill routes: /api/bills/:orderId, /api/bills/pay

3. Frontend:

   - Pages: Home (landing), Products, Product Detail, Login, Register, Profile, Orders, Invoice

   - Components: Header, Footer, ProductCard, OrderCard

   - Auth context for JWT management

4. Seed database with 12 sample products

Let me create comprehensive implementation using bulk_file_writer.

Observation: Your thought has been logged.

FOLLOW THE PROCESS

Action: $ cd /app/backend && pip install bcrypt pyjwt python-multipart && pip freeze > requirements.txt

Observation: Looking in indexes: https://pypi.org/simple, https://d33sy5i8bnduwe.cloudfront.net/simple/

Requirement already satisfied: bcrypt in /root/.venv/lib/python3.11/site-packages (4.1.3)

Requirement already satisfied: pyjwt in /root/.venv/lib/python3.11/site-packages (2.12.1)

Requirement already satisfied: python-multipart in /root/.venv/lib/python3.11/site-packages (0.0.22)

Exit code: 0

Coffee Shop Management System:

```
from fastapi import FastAPI, APIRouter, HTTPException, Depends, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from dotenv import load_dotenv
from starlette.middleware.cors import CORSMiddleware
from motor.motor_asyncio import AsyncIOMotorClient
import os
import logging
from pathlib import Path
from pydantic import BaseModel, Field, ConfigDict, EmailStr
from typing import List, Optional
import uuid
from datetime import datetime, timezone, timedelta
import bcrypt
import jwt

ROOT_DIR = Path(__file__).parent
load_dotenv(ROOT_DIR / '.env')

mongo_url = os.environ['MONGO_URL']
client = AsyncIOMotorClient(mongo_url)
db = client[os.environ['DB_NAME']]

app = FastAPI()
api_router = APIRouter(prefix="/api")
security = HTTPBearer()

JWT_SECRET = os.environ.get('JWT_SECRET', 'your-secret-key-change-in-production')
JWT_ALGORITHM = 'HS256'
```

```
# Models
class UserRegister(BaseModel):
    name: str
    email: EmailStr
    password: str
    contactNo: Optional[str] = None
    address: Optional[str] = None

class UserLogin(BaseModel):
    email: EmailStr
    password: str

class UserResponse(BaseModel):
    model_config = ConfigDict(extra="ignore")
    id: str
    name: str
    email: str
    contactNo: Optional[str] = None
    address: Optional[str] = None
    createdDate: str
    isActive: bool

class TokenResponse(BaseModel):
    token: str
    user: UserResponse

class Product(BaseModel):
    model_config = ConfigDict(extra="ignore")
    id: str
    name: str
```

```
    description: str
    price: float
    quantity: int
    category: str
    imageUrl: str
    isAvailable: bool
    createdDate: str

class OrderItemInput(BaseModel):
    productId: str
    quantity: int

class OrderCreate(BaseModel):
    items: List[OrderItemInput]
    deliveryAddress: str
    specialNotes: Optional[str] = None

class OrderItem(BaseModel):
    model_config = ConfigDict(extra="ignore")
    id: str
    productId: str
    productName: str
    quantity: int
    unitPrice: float
    lineTotal: float

class Order(BaseModel):
    model_config = ConfigDict(extra="ignore")
    id: str
    customerId: str
```

```
    orderDate: str
    subTotal: float
    tax: float
    deliveryCharge: float
    totalAmount: float
    status: str
    deliveryAddress: str
    specialNotes: Optional[str] = None
    items: List[OrderItem]

class Bill(BaseModel):
    model_config = ConfigDict(extra="ignore")
    id: str
    orderId: str
    subTotal: float
    tax: float
    discount: float
    totalAmount: float
    billDate: str
    paymentMethod: Optional[str] = None
    paymentStatus: str
    transactionId: Optional[str] = None
    paymentDate: Optional[str] = None

class PaymentRequest(BaseModel):
    paymentMethod: str

# Helper functions
def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')
```

```
def verify_password(password: str, hashed: str) -> bool:
    return bcrypt.checkpw(password.encode('utf-8'), hashed.encode('utf-8'))

def create_token(user_id: str) -> str:
    payload = {
        'user_id': user_id,
        'exp': datetime.now(timezone.utc) + timedelta(days=7)
    }
    return jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGORITHM)

async def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security)):
    try:
        token = credentials.credentials
        payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
        user_id = payload.get('user_id')
        user = await db.customers.find_one({'id': user_id}, {'_id': 0})
        if not user:
            raise HTTPException(status_code=401, detail='User not found')
        return user
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail='Token expired')
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail='Invalid token')

# Auth routes
@api_router.post('/auth/register', response_model=TokenResponse)
async def register(user_input: UserRegister):
    existing = await db.customers.find_one({'email': user_input.email}, {'_id': 0})
    if existing:
```

```
        raise HTTPException(status_code=400, detail='Email already registered')

    user_id = str(uuid.uuid4())
    hashed_pw = hash_password(user_input.password)

    user_doc = {
        'id': user_id,
        'name': user_input.name,
        'email': user_input.email,
        'password': hashed_pw,
        'contactNo': user_input.contactNo,
        'address': user_input.address,
        'createdDate': datetime.now(timezone.utc).isoformat(),
        'isActive': True
    }

    await db.customers.insert_one(user_doc)

    token = create_token(user_id)
    user_response = UserResponse(**{k: v for k, v in user_doc.items() if k != 'password'})

    return TokenResponse(token=token, user=user_response)

@api_router.post('/auth/login', response_model=TokenResponse)
async def login(credentials: UserLogin):
    user = await db.customers.find_one({'email': credentials.email}, {'_id': 0})
    if not user or not verify_password(credentials.password, user['password']):
        raise HTTPException(status_code=401, detail='Invalid email or password')

    if not user.get('isActive', True):
```

```
        raise HTTPException(status_code=403, detail='Account is inactive')

    token = create_token(user['id'])
    user_response = UserResponse(**{k: v for k, v in user.items() if k != 'password'})

    return TokenResponse(token=token, user=user_response)

@api_router.get('/auth/profile', response_model=UserResponse)
async def get_profile(current_user = Depends(get_current_user)):
    return UserResponse(**{k: v for k, v in current_user.items() if k != 'password'})

@api_router.put('/auth/profile', response_model=UserResponse)
async def update_profile(name: Optional[str] = None, contactNo: Optional[str] = None,
                         address: Optional[str] = None, current_user = Depends(get_current_user)):
    update_data = {}
    if name:
        update_data['name'] = name
    if contactNo:
        update_data['contactNo'] = contactNo
    if address:
        update_data['address'] = address

    if update_data:
        await db.customers.update_one({'id': current_user['id']}, {'$set': update_data})
        updated_user = await db.customers.find_one({'id': current_user['id']}, {'_id': 0})
        return UserResponse(**{k: v for k, v in updated_user.items() if k != 'password'})

    return UserResponse(**{k: v for k, v in current_user.items() if k != 'password'})

# Product routes
```

```
@api_router.get('/products', response_model=List[Product])
async def get_products(category: Optional[str] = None, search: Optional[str] = None):
    query = {'isAvailable': True}
    if category:
        query['category'] = category
    if search:
        query['name'] = {'$regex': search, '$options': 'i'}

    products = await db.products.find(query, {'_id': 0}).to_list(100)
    return products

@api_router.get('/products/{product_id}', response_model=Product)
async def get_product(product_id: str):
    product = await db.products.find_one({'id': product_id}, {'_id': 0})
    if not product:
        raise HTTPException(status_code=404, detail='Product not found')
    return product

# Order routes
@api_router.post('/orders', response_model=Order)
async def create_order(order_input: OrderCreate, current_user = Depends(get_current_user)):
    if not order_input.items:
        raise HTTPException(status_code=400, detail='Order must contain at least one item')

    order_id = str(uuid.uuid4())
    order_items = []
    sub_total = 0.0

    for item in order_input.items:
        product = await db.products.find_one({'id': item.productId}, {'_id': 0})
```

```
        if not product:
            raise HTTPException(status_code=404, detail=f'Product {item.productId} not found')
        if not product['isAvailable']:
            raise HTTPException(status_code=400, detail=f'Product {product["name"]} is not
available')
        if product['quantity'] < item.quantity:
            raise HTTPException(status_code=400, detail=f'Insufficient stock for {product["name"]}')

        line_total = product['price'] * item.quantity
        sub_total += line_total

        order_item = {
            'id': str(uuid.uuid4()),
            'productId': item.productId,
            'productName': product['name'],
            'quantity': item.quantity,
            'unitPrice': product['price'],
            'lineTotal': line_total
        }
        order_items.append(order_item)

        await db.products.update_one(
            {'id': item.productId},
            {'$inc': {'quantity': -item.quantity}}
        )

    tax = sub_total * 0.05
    delivery_charge = 50.0
    total_amount = sub_total + tax + delivery_charge
```

```
    order_doc = {
        'id': order_id,
        'customerId': current_user['id'],
        'orderDate': datetime.now(timezone.utc).isoformat(),
        'subTotal': sub_total,
        'tax': tax,
        'deliveryCharge': delivery_charge,
        'totalAmount': total_amount,
        'status': 'Pending',
        'deliveryAddress': order_input.deliveryAddress,
        'specialNotes': order_input.specialNotes,
        'items': order_items
    }

    await db.orders.insert_one(order_doc)

    bill_doc = {
        'id': str(uuid.uuid4()),
        'orderId': order_id,
        'subTotal': sub_total,
        'tax': tax,
        'discount': 0.0,
        'totalAmount': total_amount,
        'billDate': datetime.now(timezone.utc).isoformat(),
        'paymentMethod': None,
        'paymentStatus': 'Pending',
        'transactionId': None,
        'paymentDate': None
    }
```

```
    await db.bills.insert_one(bill_doc)

    return Order(**order_doc)

@api_router.get('/orders', response_model=List[Order])
async def get_orders(current_user = Depends(get_current_user)):
    orders = await db.orders.find({'customerId': current_user['id']}, {'_id': 0}).sort('orderDate', -
1).to_list(100)
    return orders

@api_router.get('/orders/{order_id}', response_model=Order)
async def get_order(order_id: str, current_user = Depends(get_current_user)):
    order = await db.orders.find_one({'id': order_id, 'customerId': current_user['id']}, {'_id': 0})
    if not order:
        raise HTTPException(status_code=404, detail='Order not found')
    return order

@api_router.delete('/orders/{order_id}')
async def cancel_order(order_id: str, current_user = Depends(get_current_user)):
    order = await db.orders.find_one({'id': order_id, 'customerId': current_user['id']}, {'_id': 0})
    if not order:
        raise HTTPException(status_code=404, detail='Order not found')
    if order['status'] != 'Pending':
        raise HTTPException(status_code=400, detail='Only pending orders can be cancelled')

    for item in order['items']:
        await db.products.update_one(
            {'id': item['productId']},
            {'$inc': {'quantity': item['quantity']}}
        )
```

```
    await db.orders.update_one({'id': order_id}, {'$set': {'status': 'Cancelled'}})
    return {'message': 'Order cancelled successfully'}

# Bill routes
@api_router.get('/bills/{order_id}', response_model=Bill)
async def get_bill(order_id: str, current_user = Depends(get_current_user)):
    order = await db.orders.find_one({'id': order_id, 'customerId': current_user['id']}, {'_id': 0})
    if not order:
        raise HTTPException(status_code=404, detail='Order not found')

    bill = await db.bills.find_one({'orderId': order_id}, {'_id': 0})
    if not bill:
        raise HTTPException(status_code=404, detail='Bill not found')

    return bill

@api_router.get('/bills', response_model=List[Bill])
async def get_bills(current_user = Depends(get_current_user)):
    orders = await db.orders.find({'customerId': current_user['id']}, {'_id': 0}).to_list(100)
    order_ids = [o['id'] for o in orders]
    bills = await db.bills.find({'orderId': {'$in': order_ids}}, {'_id': 0}).sort('billDate', -1).to_list(100)
    return bills

@api_router.post('/bills/{order_id}/pay', response_model=Bill)
async def pay_bill(order_id: str, payment: PaymentRequest, current_user =
Depends(get_current_user)):
    order = await db.orders.find_one({'id': order_id, 'customerId': current_user['id']}, {'_id': 0})
    if not order:
        raise HTTPException(status_code=404, detail='Order not found')
```

```
    bill = await db.bills.find_one({'orderId': order_id}, {'_id': 0})
    if not bill:
        raise HTTPException(status_code=404, detail='Bill not found')

    if bill['paymentStatus'] == 'Completed':
        raise HTTPException(status_code=400, detail='Bill already paid')

    transaction_id = f'TXN-{uuid.uuid4().hex[:12].upper()}'

    update_data = {
        'paymentMethod': payment.paymentMethod,
        'paymentStatus': 'Completed',
        'transactionId': transaction_id,
        'paymentDate': datetime.now(timezone.utc).isoformat()
    }

    await db.bills.update_one({'orderId': order_id}, {'$set': update_data})
    await db.orders.update_one({'id': order_id}, {'$set': {'status': 'Paid'}})

    updated_bill = await db.bills.find_one({'orderId': order_id}, {'_id': 0})
    return updated_bill

app.include_router(api_router)

app.add_middleware(
    CORSMiddleware,
    allow_credentials=True,
    allow_origins=os.environ.get('CORS_ORIGINS', '*').split(','),
    allow_methods=["*"],
```

```
    allow_headers=["*"],
)

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)

@app.on_event("shutdown")
async def shutdown_db_client():
    client.close()
```

.env

/app/backend/.env

```
MONGO_URL="mongodb://localhost:27017"
DB_NAME="test_database"
CORS_ORIGINS="*"
JWT_SECRET="coffee-shop-secret-key-change-in-production"
```

`/app/backend/seed_data.py`

Now let me create a seed script and implement the frontend:

Action: file_editor create /app/backend/seed_data.py --file-text "import asyncio

from motor.motor_asyncio import AsyncIOMotorClient

import os

from dotenv import load_dotenv

from pathlib import Path

```
from datetime import datetime, timezone
import uuid

ROOT_DIR = Path(__file__).parent
load_dotenv(ROOT_DIR / '.env')

mongo_url = os.environ['MONGO_URL']
client = AsyncIOMotorClient(mongo_url)
db = client[os.environ['DB_NAME']]

async def seed_products():
    existing_count = await db.products.count_documents({})
    if existing_count > 0:
        print(f\"Products already seeded ({existing_count} products found). Skipping...\")
        return

    products = [
        {
            'id': str(uuid.uuid4()),
            'name': 'Espresso',
            'description': 'Strong and rich espresso shot made from premium coffee beans',
            'price': 50.0,
            'quantity': 100,
            'category': 'Coffee',
            'imageUrl': 'https://images.unsplash.com/photo-1770349694376-
cc7b65d8305a?crop=entropy&cs=srgb&fm=jpg&ixid=M3w3NTY2Nzh8MHwxfHNlYXJjaHw0fHxhcnRpc2F
uYWwlMjBjb2ZmZWUlMjBjdXAlMjBsYXR0ZSUyMGFydHxlbnwwfHx8fDE3NzUyMDYzNDV8MA&ixlib=rb-
4.1.0&q=85',
            'isAvailable': True,
            'createdDate': datetime.now(timezone.utc).isoformat()
```

```
    },
    {
        'id': str(uuid.uuid4()),
        'name': 'Cappuccino',
        'description': 'Creamy cappuccino with perfect foam and rich espresso',
        'price': 80.0,
        'quantity': 85,
        'category': 'Coffee',
        'imageUrl': 'https://images.unsplash.com/photo-1760687510983-
68b5e2525bd4?crop=entropy&cs=srgb&fm=jpg&ixid=M3w3NTY2Nzh8MHwxfHNlYXJjaHwxfHxhcnRpc2F
uYWwlMjBjb2ZmZWUlMjBjdXAlMjBsYXR0ZSUyMGFydHxlbnwwfHx8fDE3NzUyMDYzNDV8MA&ixlib=rb-
4.1.0&q=85',
        'isAvailable': True,
        'createdDate': datetime.now(timezone.utc).isoformat()
    },
    {
        'id': str(uuid.uuid4()),
        'name': 'Latte',
        'description': 'Smooth and velvety latte with steamed milk',
        'price': 75.0,
        'quantity': 90,
        'category': 'Coffee',
        'imageUrl': 'https://images.unsplash.com/photo-1669643219978-
28a679a28c92?crop=entropy&cs=srgb&fm=jpg&ixid=M3w4NjA1NTZ8MHwxfHNlYXJjaHwyfHxjb2ZmZW
UlMjBiZWFucyUyMHJvYXN0ZWQlMjBlc3ByZXNzb3xlbnwwfHx8fDE3NzUyMDYzNDV8MA&ixlib=rb-
4.1.0&q=85',
        'isAvailable': True,
        'createdDate': datetime.now(timezone.utc).isoformat()
    },
    {
        'id': str(uuid.uuid4()),
```

'name': 'Americano',

'description': 'Classic American coffee with bold flavor',

'price': 60.0,

'quantity': 95,

'category': 'Coffee',

'imageUrl': 'https://images.unsplash.com/photo-1770349694376-cc7b65d8305a?crop=entropy&cs=srgb&fm=jpg&ixid=M3w3NTY2Nzh8MHwxfHNlYXJjaHw0fHxhcnRpc2FuYWwlMjBjb2ZmZWUlMjBjdXAlMjBsYXR0ZSUyMGFydHxlbnwwfHx8fDE3NzUyMDYzNDV8MA&ixlib=rb-4.1.0&q=85',

'isAvailable': True,

'createdDate': datetime.now(timezone.utc).isoformat()

},

{

'id': str(uuid.uuid4()),

'name': 'Chai Tea',

'description': 'Traditional Indian chai tea with aromatic spices',

'price': 40.0,

'quantity': 120,

'category': 'Tea',

'imageUrl': 'https://images.unsplash.com/photo-1760687510983-68b5e2525bd4?crop=entropy&cs=srgb&fm=jpg&ixid=M3w3NTY2Nzh8MHwxfHNlYXJjaHwxfHxhcnRpc2FuYWwlMjBjb2ZmZWUlMjBjdXAlMjBsYXR0ZSUyMGFydHxlbnwwfHx8fDE3NzUyMDYzNDV8MA&ixlib=rb-4.1.0&q=85',

'isAvailable': True,

'createdDate': datetime.now(timezone.utc).isoformat()

},

{

'id': str(uuid.uuid4()),

'name': 'Green Tea',

'description': 'Fresh and healthy green tea',

'price': 45.0,

'quantity': 110,

'category': 'Tea',

'imageUrl': 'https://images.unsplash.com/photo-1669643219978-28a679a28c92?crop=entropy&cs=srgb&fm=jpg&ixid=M3w4NjA1NTZ8MHwxfHNlYXJjaHwyfHxjb2ZmZWUlMjBiZWFucyUyMHJvYXN0ZWQlMjBlc3ByZXNzb3xlbnwwfHx8fDE3NzUyMDYzNDV8MA&ixlib=rb-4.1.0&q=85',

'isAvailable': True,

'createdDate': datetime.now(timezone.utc).isoformat()

},

{

'id': str(uuid.uuid4()),

'name': 'Chocolate Brownie',

'description': 'Rich chocolate brownie with a fudgy center',

'price': 35.0,

'quantity': 50,

'category': 'Pastry',

'imageUrl': 'https://images.unsplash.com/photo-1774758934999-66337d10f23c?crop=entropy&cs=srgb&fm=jpg&ixid=M3w3NDQ2MzR8MHwxfHNlYXJjaHwzfHxwYXN0cmllcyUyMGJha2VyeSUyMGl0ZW1zfGVufDB8fHx8MTc3NTIwNjM0NXww&ixlib=rb-4.1.0&q=85',

'isAvailable': True,

'createdDate': datetime.now(timezone.utc).isoformat()

},

{

'id': str(uuid.uuid4()),

'name': 'Vanilla Cake',

'description': 'Fresh vanilla cake slice with buttercream',

'price': 40.0,

'quantity': 45,

'category': 'Pastry',

'imageUrl': 'https://images.unsplash.com/photo-1769615020959-8a17bd77e849?crop=entropy&cs=srgb&fm=jpg&ixid=M3w3NDQ2MzR8MHwxfHNlYXJjaHwyfHxwYXN0cmllcyUyMGJha2VyeSUyMGl0ZW1zfGVufDB8fHx8MTc3NTIwNjM0NXww&ixlib=rb-4.1.0&q=85',

'isAvailable': True,

'createdDate': datetime.now(timezone.utc).isoformat()

},

{

'id': str(uuid.uuid4()),

'name': 'Blueberry Muffin',

'description': 'Delicious blueberry muffin baked fresh daily',

'price': 30.0,

'quantity': 60,

'category': 'Pastry',

'imageUrl': 'https://images.unsplash.com/photo-1774758934999-66337d10f23c?crop=entropy&cs=srgb&fm=jpg&ixid=M3w3NDQ2MzR8MHwxfHNlYXJjaHwzfHxwYXN0cmllcyUyMGJha2VyeSUyMGl0ZW1zfGVufDB8fHx8MTc3NTIwNjM0NXww&ixlib=rb-4.1.0&q=85',

'isAvailable': True,

'createdDate': datetime.now(timezone.utc).isoformat()

},

{

'id': str(uuid.uuid4()),

'name': 'Croissant',

'description': 'Buttery, flaky croissant',

'price': 35.0,

'quantity': 55,

'category': 'Pastry',

'imageUrl': 'https://images.unsplash.com/photo-1769615020959-8a17bd77e849?crop=entropy&cs=srgb&fm=jpg&ixid=M3w3NDQ2MzR8MHwxfHNlYXJjaHwyfHxwYXN0cmllcyUyMGJha2VyeSUyMGl0ZW1zfGVufDB8fHx8MTc3NTIwNjM0NXww&ixlib=rb-4.1.0&q=85',

'isAvailable': True,

'createdDate': datetime.now(timezone.utc).isoformat()

```
    },
    {
        'id': str(uuid.uuid4()),
        'name': 'Iced Coffee',
        'description': 'Chilled iced coffee perfect for hot days',
        'price': 70.0,
        'quantity': 80,
        'category': 'Coffee',
        'imageUrl': 'https://images.unsplash.com/photo-1770349694376-
cc7b65d8305a?crop=entropy&cs=srgb&fm=jpg&ixid=M3w3NTY2Nzh8MHwxfHNlYXJjaHw0fHxhcnRpc2F
uYWwlMjBjb2ZmZWUlMjBjdXAlMjBsYXR0ZSUyMGFydHxlbnwwfHx8fDE3NzUyMDYzNDV8MA&ixlib=rb-
4.1.0&q=85',
        'isAvailable': True,
        'createdDate': datetime.now(timezone.utc).isoformat()
    },
    {
        'id': str(uuid.uuid4()),
        'name': 'Mocha',
        'description': 'Chocolate coffee mix with whipped cream',
        'price': 90.0,
        'quantity': 75,
        'category': 'Coffee',
        'imageUrl': 'https://images.unsplash.com/photo-1760687510983-
68b5e2525bd4?crop=entropy&cs=srgb&fm=jpg&ixid=M3w3NTY2Nzh8MHwxfHNlYXJjaHwxfHxhcnRpc2F
uYWwlMjBjb2ZmZWUlMjBjdXAlMjBsYXR0ZSUyMGFydHxlbnwwfHx8fDE3NzUyMDYzNDV8MA&ixlib=rb-
4.1.0&q=85',
        'isAvailable': True,
        'createdDate': datetime.now(timezone.utc).isoformat()
    }
]
```

```
    await db.products.insert_many(products)
    print(f\"✅ Seeded {len(products)} products successfully\")

async def main():
    print(\"Starting database seed...\")
    await seed_products()
    print(\"✅ Database seeding complete!\")
    client.close()

if __name__ == '__main__':
    asyncio.run(main())
"
```

Observation: Create successful: /app/backend/seed_data.py

```
$ cd /app/backend && python seed_data.py
```

Action: $ cd /app/backend && python seed_data.py

Observation: Starting database seed...

✅ Seeded 12 products successfully

✅ Database seeding complete!

Exit code: 0

```
Created3 files:
```

/app/frontend/src/index.css

```
@import
url('https://fonts.googleapis.com/css2?family=Cormorant+Garamond:wght@400;500
;600;700&family=Manrope:wght@400;500;600;700;800&display=swap');

@tailwind base;
@tailwind components;
@tailwind utilities;

body {
    margin: 0;
    font-family: 'Manrope', -apple-system, BlinkMacSystemFont, 'Segoe UI',
'Roboto', sans-serif;
    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
}

code {
    font-family:
        source-code-pro, Menlo, Monaco, Consolas, 'Courier New', monospace;
}

@layer base {
    :root {
        --background: 32 10% 98%;
        --foreground: 22 28% 13%;
        --card: 0 0% 100%;
```

```
        --card-foreground: 22 28% 13%;
        --popover: 0 0% 100%;
        --popover-foreground: 22 28% 13%;
        --primary: 11 57% 52%;
        --primary-foreground: 0 0% 100%;
        --secondary: 32 16% 93%;
        --secondary-foreground: 22 28% 13%;
        --muted: 32 16% 93%;
        --muted-foreground: 22 8% 42%;
        --accent: 101 15% 51%;
        --accent-foreground: 0 0% 100%;
        --destructive: 4 62% 45%;
        --destructive-foreground: 0 0% 100%;
        --border: 32 11% 90%;
        --input: 32 11% 90%;
        --ring: 11 57% 52%;
        --radius: 0.75rem;
    }

    .dark {
        --background: 22 13% 11%;
        --foreground: 32 16% 93%;
        --card: 22 9% 15%;
        --card-foreground: 32 16% 93%;
        --popover: 22 9% 15%;
        --popover-foreground: 32 16% 93%;
        --primary: 11 52% 60%;
        --primary-foreground: 0 0% 100%;
        --secondary: 22 10% 18%;
        --secondary-foreground: 32 16% 93%;
        --muted: 22 10% 18%;
        --muted-foreground: 22 8% 62%;
        --accent: 101 12% 56%;
        --accent-foreground: 0 0% 100%;
        --destructive: 4 53% 57%;
        --destructive-foreground: 0 0% 100%;
        --border: 22 11% 22%;
        --input: 22 11% 22%;
        --ring: 11 52% 60%;
    }
}

@layer base {
    * {
        @apply border-border;
    }
    body {
        @apply bg-background text-foreground;
```

```
    }
    h1, h2, h3, h4, h5, h6 {
        font-family: 'Cormorant Garamond', serif;
    }
}

@layer base {
    [data-debug-wrapper="true"] {
        display: contents !important;
    }

    [data-debug-wrapper="true"] > * {
        margin-left: inherit;
        margin-right: inherit;
        margin-top: inherit;
        margin-bottom: inherit;
        padding-left: inherit;
        padding-right: inherit;
        padding-top: inherit;
        padding-bottom: inherit;
        column-gap: inherit;
        row-gap: inherit;
        gap: inherit;
        border-left-width: inherit;
        border-right-width: inherit;
        border-top-width: inherit;
        border-bottom-width: inherit;
        border-left-style: inherit;
        border-right-style: inherit;
        border-top-style: inherit;
        border-bottom-style: inherit;
        border-left-color: inherit;
        border-right-color: inherit;
        border-top-color: inherit;
        border-bottom-color: inherit;
    }
}
```

AuthContext.js

/app/frontend/src/context/AuthContext.js

```
import React, { createContext, useContext, useState, useEffect } from
'react';
import axios from 'axios';

const AuthContext = createContext();

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within AuthProvider');
  }
  return context;
};

const BACKEND_URL = process.env.REACT_APP_BACKEND_URL;
const API = `${BACKEND_URL}/api`;

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem('token'));
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const initAuth = async () => {
      const storedToken = localStorage.getItem('token');
      if (storedToken) {
        try {
          const response = await axios.get(`${API}/auth/profile`, {
            headers: { Authorization: `Bearer ${storedToken}` }
          });
          setUser(response.data);
          setToken(storedToken);
        } catch (error) {
          localStorage.removeItem('token');
          setToken(null);
        }
      }
      setLoading(false);
    };
    initAuth();
  }, []);
```

```
  const login = async (email, password) => {
    const response = await axios.post(`${API}/auth/login`, { email, password
});
    const { token: newToken, user: userData } = response.data;
    localStorage.setItem('token', newToken);
    setToken(newToken);
    setUser(userData);
    return userData;
  };

  const register = async (name, email, password, contactNo, address) => {
    const response = await axios.post(`${API}/auth/register`, {
      name,
      email,
      password,
      contactNo,
      address
    });
    const { token: newToken, user: userData } = response.data;
    localStorage.setItem('token', newToken);
    setToken(newToken);
    setUser(userData);
    return userData;
  };

  const logout = () => {
    localStorage.removeItem('token');
    setToken(null);
    setUser(null);
  };

  const getAuthHeaders = () => {
    return token ? { Authorization: `Bearer ${token}` } : {};
  };

  return (
    <AuthContext.Provider value={{ user, token, login, register, logout,
loading, getAuthHeaders }}>
      {children}
    </AuthContext.Provider>
  );
};
```

Header.js

/app/frontend/src/components/Header.js

```
import React from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { ShoppingBag, User, LogOut, Coffee } from 'lucide-react';
import { Button } from './ui/button';

const Header = () => {
  const { user, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate('/');
  };

  return (
    <header className="sticky top-0 z-50 backdrop-blur-xl bg-[#FAF8F5]/80
dark:bg-[#1F1A17]/80 border-b border-[#EAE4DC] dark:border-[#3F352E]">
      <div className="container mx-auto px-6 md:px-12">
        <div className="flex items-center justify-between h-16">
          <Link to="/" className="flex items-center gap-2" data-testid="logo-
link">
            <Coffee className="w-8 h-8 text-[#C05E47]" />
            <span className="text-2xl font-medium text-[#2C1E16] dark:text-
[#F3EFE9]" style={{ fontFamily: 'Cormorant Garamond, serif' }}>
              Coffee Shop
            </span>
          </Link>

          <nav className="hidden md:flex items-center gap-8">
            <Link to="/products" className="text-sm font-medium text-
[#6C5F5B] hover:text-[#C05E47] transition-colors" data-testid="products-nav-
link">
              Products
            </Link>
            {user && (
              <Link to="/orders" className="text-sm font-medium text-
[#6C5F5B] hover:text-[#C05E47] transition-colors" data-testid="orders-nav-
link">
                My Orders
              </Link>
            )}
          </nav>
```

```
          <div className="flex items-center gap-4">
            {user ? (
              <>
                <Link to="/profile" data-testid="profile-link">
                  <Button variant="ghost" size="sm" className="gap-2">
                    <User className="w-4 h-4" />
                    <span className="hidden md:inline">{user.name}</span>
                  </Button>
                </Link>
                <Button variant="ghost" size="sm" onClick={handleLogout}
className="gap-2" data-testid="logout-button">
                  <LogOut className="w-4 h-4" />
                  <span className="hidden md:inline">Logout</span>
                </Button>
              </>
            ) : (
              <>
                <Link to="/login" data-testid="login-nav-link">
                  <Button variant="ghost" size="sm">Login</Button>
                </Link>
                <Link to="/register" data-testid="register-nav-link">
                  <Button className="bg-[#C05E47] hover:bg-[#A64B36] text-
white" size="sm">
                    Sign Up
                  </Button>
                </Link>
              </>
            )}
          </div>
        </div>
      </div>
    </header>
  );
};

export default Header;
```